CNAD — Cloud & AWS Development

Complete Viva Question Bank with Answers

Covering Practicals 1 through 7 | Academic Year 2025–26

Total Questions: 191 | All Topics Covered

Topics: boto3 | CloudFormation | RDS | ElastiCache | DynamoDB | Lambda | API Gateway | Step Functions | SAM | Cognito | X-Ray | CodeBuild | CodeDeploy | CodePipeline | Athena | IAM | VPC | S3

Practical 1 — Programming AWS Services in Python (boto3)

1A · Core / Conceptual Questions

Q1: What is boto3 and how does it differ from the AWS CLI?

Ans: boto3 is the official AWS SDK (Software Development Kit) for Python. It allows Python programs to interact with AWS services programmatically. The AWS CLI is a command-line tool for humans to interact with AWS interactively via terminal commands. boto3 is used inside Python scripts/applications, while AWS CLI is used manually or in shell scripts.

Q2: What are the two main types of objects in boto3 — client and resource? How do they differ?

Ans: Client provides a low-level, 1:1 mapping to AWS service APIs. It returns raw JSON dictionaries. Resource is a higher-level, object-oriented abstraction that wraps clients and provides Python objects (e.g., ec2.Instance objects with attributes). Use client for full API access; use resource for easier object-oriented interaction.

Q3: How do you configure AWS credentials for boto3? Name at least two methods.

Ans: (1) AWS credentials file (~/.aws/credentials) configured via aws configure; (2) Environment variables (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY); (3) IAM Role attached to EC2 instance or Lambda; (4) Directly in code using boto3.Session() — least preferred.

Q4: What is an EC2 instance? Explain the difference between running, stopped, and terminated states.

Ans: EC2 is a virtual server in AWS. Running: instance is active and incurring compute charges. Stopped: instance is shut down but EBS volume is retained; you can restart it; storage charges apply. Terminated: instance is permanently deleted; data is lost; no further charges.

Q5: Write the boto3 call to list all EC2 instances and filter by state 'running'.

Ans: ec2 = boto3.client('ec2'); response = ec2.describe_instances(Filters=[{'Name': 'instance-state-name', 'Values': ['running']}]). Then iterate response['Reservations'] -> each['Instances'].

Q6: What Python method is used to start stopped EC2 instances using boto3?

Ans: ec2.start_instances(InstanceIds=[list_of_ids]). This is called on a boto3 EC2 client object and takes a list of instance IDs to start.

Q7: What is the difference between start_instances() and run_instances() in boto3?

Ans: start_instances() starts already-existing stopped EC2 instances. run_instances() launches brand-new EC2 instances (like launching from an AMI). start_instances requires existing instance IDs; run_instances creates new ones.

Q8: What is IAM? Name the four main IAM entity types.

Ans: IAM (Identity and Access Management) controls who can access what in AWS. The four main entity types are: (1) Users — individual people/applications, (2) Groups — collection of users sharing permissions, (3) Roles — assumed by services/applications for temporary access, (4) Policies — JSON documents that define permissions.

Q9: What is the difference between an IAM managed policy and an inline policy?

Ans: Managed Policy: standalone policy object that can be attached to multiple users/groups/roles; reusable (AWS-managed or customer-managed). Inline Policy: directly embedded into a single user/group/role; not reusable; deleted when the entity is deleted.

Q10: What is the AdministratorAccess policy and why is it considered dangerous?

Ans: AdministratorAccess is an AWS-managed policy that grants full access to all AWS services and resources (Action: *, Resource: *). It is dangerous because any compromised account with this policy can delete, modify, or exfiltrate any resource in the AWS account.

Q11: Explain the principle of least privilege in IAM.

Ans: The principle of least privilege means granting only the minimum permissions required for a user, role, or service to perform its intended function — nothing more. This reduces the blast radius if credentials are compromised.

Q12: What is a wildcard action (*) in an IAM policy and what risk does it pose?

Ans: Action: * means the policy allows every possible action on AWS. This effectively gives unrestricted access to all services, posing a massive security risk as a compromised identity could perform any operation including deleting resources or creating admin accounts.

Q13: What does the IAMSecurityAuditor script do when it finds an over-privileged user?

Ans: It detaches the dangerous managed policies (like AdministratorAccess), deletes inline policies containing wildcard actions, and attaches a replacement LimitedAccessPolicy that grants only read-only access to EC2 and S3, following least privilege.

Q14: What is a VPC? What problem does it solve in cloud networking?

Ans: A VPC (Virtual Private Cloud) is a logically isolated network within AWS where you launch resources. It solves the problem of resource isolation and security by letting you define your own IP ranges, subnets, routing, and firewall rules, preventing your resources from being exposed on the public internet by default.

Q15: What is a CIDR block? Explain what 10.0.0.0/16 means.

Ans: CIDR (Classless Inter-Domain Routing) block defines an IP address range. 10.0.0.0/16 means the first 16 bits are fixed (10.0.x.x), giving 65,536 IP addresses ranging from 10.0.0.0 to 10.0.255.255. /16 is the prefix length indicating how many bits are the network portion.

Q16: What is the difference between a public subnet and a private subnet in AWS?

Ans: Public subnet: has a route to an Internet Gateway (IGW), so resources can send/receive traffic from the internet. Private subnet: has no direct route to IGW; resources cannot be reached from the internet directly (use NAT Gateway for outbound only).

Q17: What is an Internet Gateway (IGW) and why must it be attached to a VPC?

Ans: An IGW is a horizontally scaled, redundant VPC component that enables communication between the VPC and the internet. It must be attached to a VPC to make that VPC internet-facing. Without it, no traffic can flow in or out from the internet regardless of routing.

Q18: What is a route table? What does the route 0.0.0.0/0 → IGW mean?

Ans: A route table contains rules (routes) that determine where network traffic is directed. The route 0.0.0.0/0 → IGW means all traffic destined for any IP address (i.e., internet traffic) is sent through the Internet Gateway — this is what makes a subnet public.

Q19: What is S3? Name the boto3 operations used for basic S3 CRUD.

Ans: S3 (Simple Storage Service) is AWS's object storage service. boto3 CRUD operations: Create — s3.put_object() or s3.upload_file(); Read — s3.get_object() or s3.download_file(); Update — s3.put_object() again with same key; Delete — s3.delete_object().

Q20: What is ElastiCache? What caching engines does it support?

Ans: Amazon ElastiCache is a fully managed in-memory caching service. It supports two engines: (1) Redis — supports rich data structures, persistence, pub/sub, and clustering; (2) Memcached — simpler, multi-threaded, pure cache with no persistence.

Q21: What is Amazon RDS? How does it differ from installing a database on EC2?

Ans: RDS (Relational Database Service) is a fully managed relational database service. AWS handles patching, backups, Multi-AZ failover, and scaling automatically. On EC2, you install and manage the DB yourself, handling all administration manually.

Q22: What is the difference between DynamoDB and RDS?

Ans: DynamoDB is a fully managed NoSQL key-value/document database — schema-less, serverless, infinitely scalable with single-digit millisecond latency. RDS is a managed SQL relational database with fixed schemas, ACID compliance, and complex query support (JOIN, transactions). Use DynamoDB for flexible, high-scale workloads; RDS for structured, relational data.

Q23: What does boto3.resource('ec2') return versus boto3.client('ec2')?

Ans: boto3.resource('ec2') returns a high-level EC2 ServiceResource object giving access to EC2 objects like ec2.Instance with attributes. boto3.client('ec2') returns a low-level client that directly maps to the EC2 API, returning raw dictionaries.

Q24: What exception type would you catch if you try to create an IAM user that already exists?

Ans: You catch botocore.exceptions.ClientError and check the error code: error['Error']['Code'] == 'EntityAlreadyExists'. This indicates the IAM user with that name already exists.

Q25: How would you list all S3 buckets using boto3?

Ans: s3 = boto3.client('s3'); response = s3.list_buckets(); buckets = [b['Name'] for b in response['Buckets']]. The list_buckets() method returns all buckets in the account.

1B · Advanced / Code-Level Questions

Q1: Explain the LimitedAccessPolicy created in the IAM auditor. What permissions does it grant and why?

Ans: LimitedAccessPolicy grants: ec2:Describe* (read-only EC2 access) and s3:GetObject, s3:ListBucket (read-only S3 access). It is created as a remediation policy to replace over-privileged policies. These minimal permissions let the user perform basic operations without destructive capabilities.

Q2: What is the difference between attach_user_policy() and put_user_policy() in boto3?

Ans: attach_user_policy() attaches a managed policy (by ARN) to a user — the policy exists as a standalone object. put_user_policy() creates/replaces an inline policy embedded directly in the user. Managed policies are reusable; inline policies are tightly coupled to the user.

Q3: How does the _has_wildcard_action() method detect dangerous inline policies?

Ans: It iterates through each inline policy document's Statement list. For each statement with Effect=Allow, it checks if the Action is either the string '*' or a list containing '*'. If found, it returns True, flagging the policy as dangerous.

Q4: What is STS in AWS? How is it used in the auditor to get the account ID for policy ARN construction?

Ans: STS (Security Token Service) provides temporary credentials and identity information. In the auditor, sts_client.get_caller_identity()['Account'] retrieves the current AWS account ID, which is required to construct the full ARN for managed policies like arn:aws:iam::ACCOUNT_ID:policy/LimitedAccessPolicy.

Q5: Why is MapPublicIpOnLaunch set to True for public subnets when creating VPC with boto3?

Ans: Setting MapPublicIpOnLaunch=True ensures that any EC2 instance launched into the public subnet automatically receives a public IP address, making it reachable from the internet without manual IP assignment.

Q6: What would happen if you called create_route() before attach_internet_gateway()?

Ans: The route 0.0.0.0/0 → IGW would fail because the IGW is not yet attached to the VPC, meaning it has no valid gateway ID to reference. The route creation would raise a ClientError (invalid gateway).

Q7: Explain the VPCInfrastructure class design — why is it structured as a class rather than standalone functions?

Ans: Using a class allows state (VPC ID, subnet IDs, IGW ID, route table ID) to be stored as instance attributes and shared across methods without passing them as parameters. This encapsulates the entire infrastructure creation workflow and makes the code reusable, testable, and maintainable.

Q8: What is CRUD? Give one boto3 example for each operation on DynamoDB.

Ans: CRUD = Create, Read, Update, Delete. Create: table.put_item(Item={...}); Read: table.get_item(Key={...}); Update: table.update_item(Key={...}, UpdateExpression='SET #n = :v', ...); Delete: table.delete_item(Key={...}).

Practical 2 — Provisioning Infrastructure Through Code (CloudFormation & Terraform)

2A · Core / Conceptual Questions

Q1: What is AWS CloudFormation? How does it relate to Infrastructure as Code (IaC)?

Ans: CloudFormation is AWS's native IaC service. It lets you define your entire AWS infrastructure in YAML/JSON template files, then automatically provisions and manages resources from those templates. IaC means treating infrastructure exactly like application code — version-controlled, repeatable, and automated.

Q2: What is a CloudFormation template? What are its main sections?

Ans: A CloudFormation template is a YAML or JSON file describing AWS resources to create. Main sections: AWSTemplateFormatVersion, Description, Parameters, Mappings, Conditions, Resources (required), Outputs.

Q3: What is the difference between a root stack and a nested (child) stack?

Ans: Root stack is the top-level stack deployed directly. Nested (child) stacks are created by the root stack using AWS::CloudFormation::Stack resources. Child stacks allow modular template design, breaking complex infrastructure into reusable components.

Q4: Why must child stack templates be stored in S3 for nested stacks?

Ans: CloudFormation cannot reference local files; it needs a URL to fetch the template. S3 provides a stable, accessible URL (TemplateURL) for child stack templates so CloudFormation can retrieve and deploy them during stack creation.

Q5: What is the purpose of the Outputs section in a CloudFormation template?

Ans: Outputs expose values from the stack (like VPC ID, subnet IDs) so they can be referenced by other stacks via cross-stack references or nested stack outputs. They also display important resource identifiers in the AWS Console.

Q6: What is a CloudFormation export? How do you export a value from a stack?

Ans: An export makes an output value available for import by other stacks (using !ImportValue). To export: in Outputs, add Export: Name: <unique-name> under the output value.

Q7: What does !GetAtt NetworkStack.Outputs.VpcId do in the root stack?

Ans: It retrieves the VpcId output value from the nested NetworkStack resource. !GetAtt (GetAttribute) on a nested stack resource allows the root stack to access outputs from child stacks and pass them to other child stacks.

Q8: What does !Sub do in CloudFormation? Give an example.

Ans: !Sub substitutes variables in a string. Example: !Sub 'https://s3.amazonaws.com/${AWS::StackName}/template.yaml' — it replaces ${AWS::StackName} with the actual stack name at deployment time.

Q9: What does !Ref do in CloudFormation? Give an example.

Ans: !Ref returns the value of a parameter or the default identifier of a resource. Example: !Ref VpcId returns the value of the VpcId parameter; !Ref MyInstance returns the EC2 instance's physical resource ID.

Q10: What is a NAT Gateway? Why is it placed in the public subnet?

Ans: A NAT Gateway allows private subnet resources to initiate outbound internet connections (e.g., downloading packages) while preventing inbound connections from the internet. It is placed in the public subnet because it needs internet access (via IGW) to forward traffic on behalf of private resources.

Q11: What is an Elastic IP? Why does the NAT Gateway require one?

Ans: An Elastic IP is a static, public IPv4 address. NAT Gateway requires one because it needs a fixed public IP address to send outbound traffic to the internet. Without a static IP, the source address would change and external services couldn't whitelist it.

Q12: What is a Security Group? How does it differ from a Network ACL?

Ans: Security Group: stateful, instance-level firewall — if inbound is allowed, return traffic is automatically allowed. Network ACL: stateless, subnet-level firewall — you must explicitly allow both inbound and outbound rules. SGs are applied to instances; NACLs to subnets.

Q13: What is SourceSecurityGroupId in a security group rule? Why is it used for WebServerSG?

Ans: SourceSecurityGroupId restricts inbound traffic to only those resources belonging to the specified security group. For WebServerSG, it only allows traffic from LoadBalancerSG — meaning web servers only accept connections coming through the load balancer, blocking direct internet access to web servers.

Q14: What is DependsOn used for in CloudFormation? Give an example from the practical.

Ans: DependsOn explicitly declares that one resource must be created before another, overriding CloudFormation's automatic dependency detection. Example: NAT Gateway DependsOn VPCGatewayAttachment, ensuring the IGW is attached to the VPC before NAT is created.

Q15: What status should all three stacks reach after successful deployment?

Ans: All three stacks (RootStack, NetworkStack, SecurityStack) should reach CREATE_COMPLETE status, indicating all resources were provisioned successfully without errors.

Q16: What is the difference between public and private route tables in this practical?

Ans: Public route table has a route 0.0.0.0/0 → IGW, enabling internet access for public subnets. Private route table has a route 0.0.0.0/0 → NAT Gateway, allowing private subnets to make outbound internet requests while remaining unreachable from the internet.

Q17: What ports does the LoadBalancerSG allow inbound? What is the source CIDR?

Ans: LoadBalancerSG allows inbound on port 80 (HTTP) and port 443 (HTTPS) from 0.0.0.0/0, meaning it accepts traffic from anywhere on the internet.

Q18: Why does WebServerSG NOT use a CIDR range as the traffic source?

Ans: Using a security group ID as source (LoadBalancerSG) is more secure than a CIDR range. It ensures only traffic from the load balancer reaches the web servers, not from any arbitrary IP. If the LB IP changes, the rule still works without updating.

Q19: What is the VPCGatewayAttachment resource type? What does it do?

Ans: AWS::EC2::VPCGatewayAttachment attaches an Internet Gateway to a VPC. Without this attachment, the IGW exists but is not associated with any VPC, so no internet traffic can flow — route table rules pointing to the IGW would be invalid.

Q20: What CloudFormation intrinsic function is used to pass the VPC ID from NetworkStack to SecurityStack?

Ans: !GetAtt is used — specifically !GetAtt NetworkStack.Outputs.VpcId — to retrieve the VpcId output from the NetworkStack nested resource and pass it as a parameter to the SecurityStack.

2B · Advanced Questions

Q1: What is a CloudFormation change set? When would you use it?

Ans: A change set is a preview of what changes CloudFormation will make to a stack before actually applying them. Use it before updating a live stack to verify which resources will be added, modified, or deleted, preventing accidental disruption.

Q2: How does CloudFormation determine the creation order of NetworkStack and SecurityStack automatically?

Ans: CloudFormation analyzes resource references in the template. Since SecurityStack references !GetAtt NetworkStack.Outputs.VpcId, it automatically knows NetworkStack must be created first — the dependency is implicit from the reference.

Q3: What is the difference between CloudFormation cross-stack references and nested stack outputs?

Ans: Cross-stack references use Export/ImportValue across independent stacks (stacks have separate lifecycles). Nested stack outputs use !GetAtt ParentStack.Outputs.Key within a parent-child relationship — child stacks are owned by the parent and deleted with it.

Q4: What happens if you delete the root stack — are the child stacks also deleted?

Ans: Yes. Deleting the root stack triggers deletion of all nested child stacks and their resources in reverse dependency order. Child stacks are lifecycle-coupled to the parent (root) stack.

Q5: What is Terraform? How does it compare to CloudFormation in terms of state management?

Ans: Terraform is an open-source IaC tool by HashiCorp supporting multi-cloud (AWS, Azure, GCP). It uses state files (.tfstate) to track deployed resources. CloudFormation manages state internally on the AWS side. Terraform's state must be managed explicitly (e.g., stored in S3); CloudFormation handles it automatically.

Q6: What is the difference between TemplateURL (S3) and inline template body in CloudFormation?

Ans: TemplateURL references a template stored in S3 (supports up to 1 MB). Inline template body is embedded in the API call (limited to 51,200 bytes). For nested stacks, only TemplateURL is supported.

Q7: What is the AWSTemplateFormatVersion field in a template and what is its current valid value?

Ans: AWSTemplateFormatVersion declares the CloudFormation template format version. The only currently valid value is '2010-09-09'. It is optional but recommended for explicitness.

Q8: What is CloudFormation drift detection? Why is it useful?

Ans: Drift detection checks whether actual resource configurations match the expected state in the CloudFormation template. It is useful for identifying unauthorized manual changes to resources that were originally provisioned by CloudFormation, ensuring infrastructure consistency.

Practical 3 — Serverless Application Development (Lambda + RDS + ElastiCache)

3A · Core / Conceptual Questions

Q1: What is Amazon RDS? What database engines does it support?

Ans: Amazon RDS (Relational Database Service) is a fully managed relational database. Supported engines: MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, and Amazon Aurora (MySQL/PostgreSQL compatible).

Q2: What is Amazon ElastiCache? What is the cache-aside (lazy loading) pattern?

Ans: ElastiCache is a managed in-memory caching service. Cache-aside (lazy loading): application first checks cache; if data exists (cache hit), return it; if not (cache miss), fetch from database, store in cache, then return. This reduces DB load.

Q3: What is a DB subnet group in RDS? Why do you need at least two subnets in different AZs?

Ans: A DB subnet group is a collection of subnets designated for RDS instances within a VPC. At least two subnets in different AZs are required because RDS needs multi-AZ capability for high availability and failover support, even for single-AZ deployments.

Q4: Why should RDS be placed in a private subnet with Public access set to No?

Ans: RDS contains sensitive data and should never be directly accessible from the internet. Placing it in a private subnet with Public access=No ensures only resources within the VPC (like app servers) can connect, protecting against external attacks.

Q5: What is Redis? On which port does it listen by default?

Ans: Redis is an open-source, in-memory data structure store supporting strings, hashes, lists, sets, sorted sets, and more. It listens on port 6379 by default.

Q6: What is MySQL? On which port does it listen by default?

Ans: MySQL is an open-source relational database management system using SQL. It listens on port 3306 by default.

Q7: Why does rds-sg use App-Server-SG as the source instead of 0.0.0.0/0?

Ans: Using App-Server-SG as source restricts database access to only the application servers (those in App-Server-SG). This implements network segmentation — only trusted app-tier instances can connect to the database, blocking all other traffic including internet sources.

Q8: What is the cache-aside pattern? How does it reduce database load?

Ans: Cache-aside: app checks Redis first; on hit, returns cached data instantly; on miss, queries MySQL, stores result in Redis with TTL, returns data. Reduces DB load because repeated reads for the same data are served from fast in-memory cache rather than hitting the database each time.

Q9: What is an RDS endpoint? How is it used in Python to connect to the database?

Ans: An RDS endpoint is the hostname/URL for connecting to the database instance, e.g., mydb.xxxxx.us-east-1.rds.amazonaws.com. In Python (pymysql): pymysql.connect(host=endpoint, user='admin', password='pass', database='mydb', port=3306).

Q10: What is the purpose of the NAT Gateway in this practical for the private subnets?

Ans: Private subnets have no direct internet access. The NAT Gateway (in the public subnet) allows resources in private subnets (RDS, ElastiCache) to make outbound requests — e.g., to download updates or reach AWS APIs — without being exposed to inbound internet traffic.

Q11: What is MSCK REPAIR TABLE in Athena? When is it needed?

Ans: MSCK REPAIR TABLE scans S3 for new partitions that exist as folders but are not yet registered in the Glue Data Catalog. It is needed when new partition folders are added to S3 manually (not via Glue Crawler or INSERT INTO), so Athena can discover and query the new data.

Q12: What is an Athena external table? How does it differ from a managed table?

Ans: An Athena external table stores data in S3 (specified by LOCATION). The metadata (schema) is in Glue Catalog but the data remains in S3. Dropping an external table removes only the metadata, not the S3 data. A managed table would store and own the data, deleting it on DROP.

Q13: What Python library is used to connect to MySQL from an EC2 instance?

Ans: PyMySQL (import pymysql) is commonly used. It is a pure-Python MySQL client library compatible with boto3 environments. Alternatively, mysql-connector-python can be used.

Q14: What is the difference between RDS encryption at rest and encryption in transit?

Ans: Encryption at rest: data stored on disk (EBS volumes, snapshots, backups) is encrypted using AWS KMS keys. Encryption in transit: data moving between application and RDS is encrypted using SSL/TLS. Both can be enabled simultaneously for full protection.

Q15: What is an RDS subnet group and how many AZs must it span at minimum?

Ans: An RDS subnet group defines which subnets RDS can use when creating DB instances in a VPC. It must span at least two Availability Zones to support Multi-AZ deployments and automatic failover capabilities.

3B · Advanced Questions

Q1: Explain the cache-aside pattern step by step. What happens on a cache miss?

Ans: 1) App receives request. 2) App checks Redis for cached data. 3) Cache hit: return data from Redis immediately. 4) Cache miss: query MySQL for data. 5) Store result in Redis with a TTL. 6) Return data to app. On miss, there is a latency penalty for the DB call, but subsequent requests hit cache.

Q2: What is a cache TTL (time-to-live)? Why is it important to set it appropriately?

Ans: TTL defines how long cached data remains valid before expiry. Too short: cache expires quickly, causing frequent DB hits (low cache hit ratio). Too long: stale data is served to users. Appropriate TTL balances freshness and performance based on how often the data changes.

Q3: What are the performance benefits of using ElastiCache in front of RDS?

Ans: In-memory reads are orders of magnitude faster than disk-based DB queries (microseconds vs milliseconds). Reduces read load on RDS, lowering costs and preventing DB overload during traffic spikes. Allows RDS to handle writes while ElastiCache handles repetitive reads.

Q4: What is cache stampede (thundering herd)? How can it be prevented?

Ans: Cache stampede occurs when many requests simultaneously experience a cache miss (e.g., after TTL expiry), all hitting the DB at once. Prevention: use probabilistic early expiration (PER), mutex locks on cache writes, or staggered TTLs to spread expiry times.

Q5: What is the difference between Redis and Memcached in ElastiCache?

Ans: Redis: supports complex data types (lists, sets, hashes), persistence (AOF/RDB), pub/sub, clustering, Multi-AZ replication. Memcached: simple key-value strings, multi-threaded, no

persistence, no replication — pure cache. Choose Redis for rich features; Memcached for simple, high-throughput caching.

Q6: What is Multi-AZ deployment in RDS? How does it differ from a Read Replica?

Ans: Multi-AZ: synchronous standby replica in another AZ for automatic failover (HA). Read Replica: asynchronous copy for offloading read traffic (scalability). Multi-AZ is for availability; Read Replica is for performance. Failover is automatic for Multi-AZ; Read Replicas require manual promotion.

Q7: Why is gp2 storage used for the RDS instance in this practical?

Ans: gp2 (General Purpose SSD) provides a cost-effective balance of performance and cost suitable for development/test workloads. It provides baseline 3 IOPS/GB with burst capability, which is sufficient for lab/learning environments without the higher cost of io1 provisioned IOPS.

Practical 4 — API Development and DynamoDB (boto3 Deep Dive)

4A · Core / Conceptual Questions

Q1: What is Amazon DynamoDB? Is it SQL or NoSQL? What data model does it use?

Ans: DynamoDB is AWS's fully managed NoSQL database. It uses a key-value and document data model. Items are schema-less JSON-like documents stored in tables identified by primary keys. It offers single-digit millisecond latency at any scale.

Q2: What is a partition key in DynamoDB? What is a sort key? Why use a composite key?

Ans: Partition key: primary attribute used to distribute data across partitions (hash key). Sort key: secondary attribute that orders items within a partition. Composite key (partition + sort) allows multiple items with the same partition key but different sort keys, enabling range queries and better data modeling.

Q3: What is a Global Secondary Index (GSI) in DynamoDB? How does it differ from the primary key?

Ans: A GSI is an alternate index with a different partition key (and optional sort key) that allows querying on non-primary key attributes. Unlike the primary key index, a GSI is eventually consistent by default and has its own provisioned throughput. You can have up to 20 GSIs per table.

Q4: What is the StatusIndex GSI created in this practical? What query does it enable?

Ans: StatusIndex uses Status as partition key and OrderDate as sort key. It enables querying: 'find all orders with Status=pending sorted by date' — something impossible with the main table's CustomerID + OrderDate primary key.

Q5: What is optimistic locking? How is it implemented using the Version attribute in DynamoDB?

Ans: Optimistic locking assumes conflicts are rare and only checks at write time. In DynamoDB, a Version attribute (number) is stored with each item. On update, a ConditionExpression checks the current Version matches what was read. If another writer updated it first (Version changed), the condition fails and the update is rejected — preventing lost updates.

Q6: What is a ConditionExpression in DynamoDB? Give an example from the practical.

Ans: ConditionExpression adds a conditional check to write operations (put, update, delete). If the condition is false, the operation fails. Example: attribute_not_exists(CustomerID) — only inserts if the item doesn't already exist, preventing duplicate customer creation.

Q7: What is put_item() vs update_item() in DynamoDB boto3?

Ans: put_item() creates a new item or completely replaces an existing item with the same key. update_item() modifies only specified attributes of an existing item, leaving other attributes unchanged. Use update_item for partial updates to avoid overwriting data.

Q8: What is batch_writer in boto3? What is the maximum items per batch request?

Ans: batch_writer is a context manager that automatically batches multiple put_item and delete_item operations efficiently. Maximum: 25 items per batch_write_item request, with each item up to 400 KB.

Q9: What is transact_write_items? How does it differ from batch_writer?

Ans: transact_write_items executes multiple write operations atomically (all-or-nothing ACID transaction) across up to 100 items and multiple tables. batch_writer batches writes for efficiency but is not atomic — individual operations can succeed or fail independently.

Q10: What is the difference between On-Demand and Provisioned capacity mode in DynamoDB?

Ans: On-Demand: auto-scales instantly; no capacity planning needed; pay-per-request. Provisioned: you specify read/write capacity units (RCUs/WCUs); lower cost for predictable workloads; can enable Auto Scaling. On-Demand is better for unpredictable traffic; Provisioned for steady workloads.

Q11: What is a DynamoDB scan? How does it differ from a query?

Ans: Scan reads every item in the entire table and filters results — very expensive and slow for large tables. Query uses the partition key (and optionally sort key) to efficiently retrieve a subset of items using indexes. Always prefer Query over Scan in production.

Q12: What is pagination in DynamoDB? What key is used to get the next page of results?

Ans: DynamoDB returns a maximum of 1 MB of data per call. If there is more data, the response includes LastEvaluatedKey. To get the next page, pass this key as ExclusiveStartKey in the subsequent request. Continue until LastEvaluatedKey is absent.

Q13: What is the LastEvaluatedKey in a DynamoDB response?

Ans: LastEvaluatedKey is the primary key of the last item evaluated in a Query/Scan that exceeded the 1 MB limit. Pass it as ExclusiveStartKey in the next call to continue from where the previous call left off, implementing pagination.

Q14: What is the maximum item size in DynamoDB?

Ans: The maximum item size in DynamoDB is 400 KB, including attribute names and values. For larger data, store the data in S3 and keep only the S3 reference in DynamoDB.

Q15: What does ACID stand for? Does DynamoDB support ACID transactions?

Ans: ACID = Atomicity, Consistency, Isolation, Durability. Yes, DynamoDB supports ACID transactions through TransactWriteItems and TransactGetItems (up to 100 items across tables). Regular operations are not ACID — only transactional API calls provide these guarantees.

4B · Advanced Questions

Q1: Why does the practical use attribute_not_exists(CustomerID) in the put_item ConditionExpression?

Ans: This prevents accidentally overwriting an existing customer record. If a customer with that CustomerID already exists, the condition fails and put_item throws ConditionalCheckFailedException, ensuring idempotent-safe creation without duplicate data.

Q2: What is adaptive retry in boto3? What does exponential backoff mean?

Ans: Adaptive retry automatically retries failed AWS API calls (e.g., due to throttling). Exponential backoff increases the wait time between retries exponentially (e.g., 1s, 2s, 4s, 8s) plus random jitter, reducing load on the service and avoiding retry storms.

Q3: What is a DynamoDB hot partition? How can it be avoided through good key design?

Ans: A hot partition occurs when too many requests hit the same partition key, overwhelming that partition's throughput limit. Avoid by: choosing high-cardinality partition keys (like UUID), adding random suffix/prefix to keys, using write sharding, or designing composite keys that distribute load.

Q4: What is a projection type in a GSI? Explain INCLUDE vs ALL vs KEYS_ONLY.

Ans: KEYS_ONLY: GSI stores only primary key + GSI key attributes (smallest, cheapest). INCLUDE: stores keys plus specified extra attributes. ALL: stores all item attributes in the GSI (most expensive but no additional table reads). Choose based on what attributes queries need.

Q5: What is the read/write cost difference between a DynamoDB transaction and a regular operation?

Ans: Transactional operations (TransactWriteItems, TransactGetItems) consume 2x the capacity units of equivalent non-transactional operations due to the overhead of two-phase commit protocol for ACID guarantees.

Q6: What is DynamoDB Streams? How could it be used with Lambda?

Ans: DynamoDB Streams captures a time-ordered sequence of item-level changes (insert, update, delete) in a table. Lambda can be triggered by stream events to implement: real-time data processing, cross-region replication, sending notifications on changes, or maintaining search indexes.

Q7: How does ExpressionAttributeNames help when the attribute name conflicts with a reserved word?

Ans: DynamoDB has reserved keywords (like Name, Status, Year). Using them directly in expressions causes errors. ExpressionAttributeNames provides placeholder substitutions: {'#st': 'status'} lets you use #st in UpdateExpression instead of the reserved word status.

Practical 5 — Serverless Orders API (Lambda + API Gateway + DynamoDB)

5A · Core / Conceptual Questions

Q1: What is AWS Lambda? What is serverless computing?

Ans: AWS Lambda is a serverless compute service that runs code in response to events without managing servers. Serverless computing means you focus only on code; AWS handles infrastructure, scaling, patching, and availability. You pay only for execution time (per 100ms).

Q2: What is a Lambda function handler? What are its two parameters?

Ans: The handler is the Python function Lambda calls when triggered. It takes two parameters: (1) event — a dict containing the trigger data (API Gateway request, S3 event, etc.); (2) context — a LambdaContext object with runtime info (function name, remaining time, request ID).

Q3: What is AWS API Gateway? What is the difference between REST API and HTTP API?

Ans: API Gateway is a fully managed service for creating and publishing APIs. REST API: supports full REST features, usage plans, custom domains, and request/response transformations. HTTP API: lighter, faster, cheaper — supports Lambda Proxy integration and JWT authorizers but fewer features.

Q4: What is Lambda Proxy Integration in API Gateway? What does it pass to the Lambda function?

Ans: Lambda Proxy Integration passes the entire HTTP request as a structured JSON event to Lambda, including: httpMethod, path, headers, queryStringParameters, pathParameters, and body. Lambda must return a formatted response with statusCode, headers, and body.

Q5: What HTTP methods are implemented in the Orders API? Map each to a CRUD operation.

Ans: GET (Read all orders — list), GET with order_id (Read single), POST (Create), PUT/PATCH (Update), DELETE (Delete). CRUD: Create=POST, Read=GET, Update=PUT, Delete=DELETE.

Q6: What is the httpMethod field in the Lambda event object?

Ans: event['httpMethod'] contains the HTTP method of the incoming request (GET, POST, PUT, DELETE, etc.). The Lambda handler uses this to route to the appropriate logic (if/elif branches) to handle each operation.

Q7: What is CORS? Why must it be enabled on the API Gateway resource?

Ans: CORS (Cross-Origin Resource Sharing) allows browsers to make requests to a different domain than the page's origin. API Gateway must enable CORS to add Access-Control-Allow-Origin headers so web browsers don't block API responses when the frontend and backend are on different domains.

Q8: What HTTP status code is returned for a successful POST (create) operation? Why 201 and not 200?

Ans: 201 Created is returned for successful resource creation. 200 OK means the request succeeded but is generic. 201 specifically communicates that a new resource was created, following HTTP standards and RESTful API best practices.

Q9: Why must both order_id AND timestamp be provided when updating or deleting an order?

Ans: The Orders table uses a composite primary key: order_id (partition key) + timestamp (sort key). DynamoDB requires ALL parts of the primary key to uniquely identify an item. Without both, DynamoDB cannot locate the specific item to update or delete.

Q10: What is the Decimal type issue in DynamoDB and how is it handled in the Lambda response?

Ans: DynamoDB returns numeric values as Python Decimal objects (not int/float), which are not JSON serializable. To fix this, use a custom JSON encoder: class DecimalEncoder(json.JSONEncoder): def default(self, o): if isinstance(o, Decimal): return float(o). Pass it to json.dumps(data, cls=DecimalEncoder).

Q11: What is a Lambda execution role? Which policy was attached in this practical?

Ans: A Lambda execution role is an IAM role that Lambda assumes when running. It defines what AWS services Lambda can access. In this practical, AmazonDynamoDBFullAccess was attached, allowing Lambda to perform all CRUD operations on DynamoDB.

Q12: What is a deployment stage in API Gateway? What stage name was used?

Ans: A deployment stage is a named reference to a snapshot of the API (e.g., dev, staging, prod). It creates the base URL for invoking the API. In this practical, the stage name was 'prod'.

Q13: What is an Invoke URL in API Gateway?

Ans: The Invoke URL is the public URL used to call the API Gateway endpoints. Format: https://{api-id}.execute-api.{region}.amazonaws.com/{stage}/{resource}. All HTTP requests to the API use this URL.

Q14: What does table.scan() return? Why is it not ideal for production?

Ans: table.scan() reads every item in the entire DynamoDB table. It is not ideal for production because it consumes full read capacity, becomes slow and expensive as the table grows, and has a 1 MB response limit requiring pagination. Use Query with indexes instead.

Q15: What would happen if AmazonDynamoDBFullAccess was not attached to the Lambda execution role?

Ans: Lambda would receive an AccessDeniedException from DynamoDB when trying to read or write. The API would return 500 errors. This is why the execution role must have appropriate DynamoDB permissions before the function can operate.

5B · Advanced Questions

Q1: What is the difference between Lambda Proxy integration and Lambda custom integration?

Ans: Proxy integration: API Gateway passes the raw request to Lambda as-is; Lambda must format the full response (statusCode, headers, body). Custom integration: API Gateway transforms the request before sending to Lambda (using mapping templates) and transforms Lambda's response before returning to client. Proxy is simpler; custom offers more control.

Q2: What is API Gateway throttling? How can it be configured?

Ans: Throttling limits the rate of API requests to protect backends from overload. Configure via: Default throttling (account-wide: 10,000 req/s burst, 5,000 req/s steady state) or per-stage/per-method limits in usage plans. Throttled requests receive HTTP 429 Too Many Requests.

Q3: What is a Lambda cold start? How can it be reduced?

Ans: Cold start occurs when Lambda initializes a new execution environment (downloads code, sets up runtime) before running the handler — adding latency (100ms–1s+). Reduce with: Provisioned Concurrency (pre-warm containers), smaller deployment packages, using runtimes with faster startup (Python, Node.js), or keeping functions warm with scheduled pings.

Q4: What is an API Gateway usage plan and API key?

Ans: A usage plan defines throttling limits and quotas for API consumers. API keys are identifiers distributed to API clients that API Gateway validates. Together they enable rate-limiting different clients, monetizing APIs, and tracking usage per consumer.

Q5: What is the difference between REST API and WebSocket API in API Gateway?

Ans: REST API is request-response: client sends request, gets response, connection closes. WebSocket API maintains persistent bidirectional connections, enabling real-time communication (chat, live updates, gaming) where server can push data to client without client polling.

Q6: Why is using ExpressionAttributeNames={'#st': 'status'} necessary in the update_item call?

Ans: 'status' is a reserved word in DynamoDB expression syntax. Using it directly in UpdateExpression causes a parsing error. ExpressionAttributeNames maps '#st' → 'status', allowing safe use of reserved attribute names in expressions.

Q7: What is the maximum timeout for a Lambda function? What is the default?

Ans: Maximum timeout: 15 minutes (900 seconds). Default: 3 seconds. If a Lambda function runs longer than its configured timeout, it is terminated and returns a Task timed out error.

Q8: How would you add authentication to the Orders API? Name two methods.

Ans: (1) Amazon Cognito Authorizer — users authenticate with Cognito and get JWT tokens; API Gateway validates tokens before routing requests. (2) Lambda Authorizer (custom authorizer) — a Lambda function validates tokens/API keys and returns an IAM policy. Also possible: IAM authentication with SigV4 signing.

Practical 6 — Advanced Topics (Step Functions, SAM, Cognito, X-Ray, Athena)

6A · Core / Conceptual Questions

Q1: What is AWS Step Functions? What is a state machine?

Ans: AWS Step Functions is a serverless orchestration service for coordinating multiple AWS services into workflows. A state machine is a series of states (steps) defined in Amazon States Language (ASL), where each state performs a task, makes a decision, waits, or handles errors.

Q2: What is Amazon States Language (ASL)? What format is it written in?

Ans: ASL is the JSON-based language used to define Step Functions state machines. It describes states, transitions, error handling, and retries. It is written in JSON format (not YAML) and defines the entire workflow structure.

Q3: What are the main state types in AWS Step Functions?

Ans: Task: executes work (Lambda, API call, Athena query); Choice: branching logic (if/else); Wait: pause for duration or timestamp; Pass: passes input to output unchanged; Succeed: ends execution successfully; Fail: ends execution with error; Map: iterate over array; Parallel: run branches concurrently.

Q4: What is the difference between a Standard workflow and an Express workflow in Step Functions?

Ans: Standard: exactly-once execution, up to 1 year duration, auditable history, higher cost — for long-running critical workflows. Express: at-least-once execution, up to 5 minutes, higher throughput, lower cost — for high-volume, short-duration event processing.

Q5: What is Amazon Athena? What type of queries does it run?

Ans: Athena is a serverless interactive query service that runs standard SQL queries directly on data stored in S3. No infrastructure to manage; pay per query (per TB scanned). Uses Presto engine and integrates with AWS Glue Data Catalog for metadata.

Q6: What is an Athena external table? Where is the actual data stored?

Ans: An Athena external table defines the schema and format (CSV, JSON, Parquet) of data while the actual data resides in S3. The table definition (metadata) is stored in Glue Data Catalog. Dropping the table removes only metadata; S3 data is unaffected.

Q7: What is the AWS Glue Data Catalog? How does Athena use it?

Ans: Glue Data Catalog is a centralized metadata repository storing database and table definitions (schemas, S3 locations, formats) for data lake assets. Athena queries Glue Catalog to understand table structure before executing SQL on S3 data — it's the metadata layer between SQL and raw S3 files.

Q8: What is S3 partitioning? How does the folder structure year=2024/month=01/ act as a partition?

Ans: Partitioning organizes S3 data into folder hierarchies that Athena treats as partition keys. When querying WHERE year=2024 AND month=01, Athena scans only the matching folder, skipping all other data — dramatically reducing scan size, cost, and query time.

Q9: What does MSCK REPAIR TABLE do in Athena?

Ans: MSCK REPAIR TABLE discovers and registers partition folders that exist in S3 but are not yet in the Glue Catalog metadata. It syncs the catalog with S3 so Athena can query newly added partition data.

Q10: What is the .sync suffix in arn:aws:states:::athena:startQueryExecution.sync?

Ans: The .sync integration pattern makes Step Functions wait for the Athena query to complete before transitioning to the next state. Without .sync, Step Functions only starts the Athena query and immediately moves on — the query runs asynchronously and results may not be ready.

Q11: What is AWS SAM? How does it simplify serverless application deployment?

Ans: AWS SAM (Serverless Application Model) is an open-source framework that extends CloudFormation with simplified syntax for serverless resources (Lambda, API Gateway, DynamoDB). SAM transforms (sam build, sam deploy) transpile SAM templates into full CloudFormation templates, reducing boilerplate significantly.

Q12: What is AWS Cognito? What is identity federation?

Ans: Amazon Cognito provides user authentication and authorization for web/mobile apps. Identity federation (via Identity Pools) allows users authenticated by external IdPs (Google, Facebook, SAML, OIDC) to receive temporary AWS credentials to access AWS services directly.

Q13: What is AWS X-Ray? How does it help with Lambda debugging?

Ans: X-Ray is a distributed tracing service that tracks requests as they flow through AWS services. For Lambda, it shows execution time per function, downstream service latencies, errors, and bottlenecks — providing end-to-end visibility for debugging performance issues and errors.

Q14: What is the ResultPath field in an ASL state definition?

Ans: ResultPath specifies where to store the state's output in the input JSON. $.queryResult stores the Athena result at the queryResult key. Using $.$ replaces the entire input; using null discards the output and passes the original input unchanged.

Q15: What is a Retry block in ASL? What is BackoffRate?

Ans: Retry automatically retries a failed state. BackoffRate is the multiplier for the wait between retries (exponential backoff). Example: IntervalSeconds=2, MaxAttempts=3, BackoffRate=2 means retries after 2s, 4s, 8s before giving up.

Q16: What is a Catch block in ASL? When is it triggered?

Ans: Catch handles errors that persist after all retries are exhausted. It defines fallback states to transition to on specific errors (or States.ALL for any error). Triggered when the Task fails and retry attempts are used up.

Q17: What is the WorkflowFailed state type used in the practical?

Ans: WorkflowFailed is a Fail state type in ASL. It terminates the execution as failed with a custom Error and Cause message, making it clear why the workflow ended — useful for error reporting and CloudWatch alerting.

Q18: What is the difference between Pass and Task state types?

Ans: Pass state: passes its input to output (optionally injecting fixed Result data) without performing any computation — useful for injecting static data or debugging. Task state: performs actual work by invoking a resource (Lambda, Athena, SQS, etc.) and waits for a result.

Q19: What is a WorkGroup in Athena?

Ans: A WorkGroup is a logical grouping of Athena queries that enforces settings like output location, encryption, data usage controls, and query result reuse. WorkGroups isolate query environments (e.g., dev vs prod) and allow per-group cost tracking.

Q20: What is OutputLocation in Athena query configuration? Why is it required?

Ans: OutputLocation specifies the S3 path where Athena writes query results (CSV files). It is required because Athena is serverless — it has nowhere to store results except S3. Without a valid S3 output location, the query cannot execute.

6B · Advanced Questions

Q1: Explain the four states in the AthenaOrdersWorkflow and why they execute in that specific order.

Ans: 1) CreateTable: creates the external Athena table pointing to S3 data (must come first so the table exists). 2) RepairTable: runs MSCK REPAIR TABLE to discover S3 partitions (must come after table creation). 3) QueryOrders: executes the business SQL query (must come after partitions are loaded). 4) QueryComplete: Pass/Succeed state confirming workflow succeeded (must be last, after results are ready).

Q2: What is the difference between polling and the .sync integration pattern in Step Functions?

Ans: Polling: Step Functions starts the task, then repeatedly polls for completion — inefficient and wastes capacity. .sync (optimistic locking callback pattern): Step Functions starts the task and waits for a callback/event when done — more efficient, no polling overhead, faster state transitions.

Q3: What is AWS SAM CLI? How does sam deploy differ from uploading a ZIP to Lambda manually?

Ans: SAM CLI is a command-line tool for building and deploying SAM applications. sam deploy: packages Lambda code, uploads to S3, transforms SAM template to CloudFormation, creates/updates the full stack atomically. Manual ZIP upload: only updates the function code, not infrastructure — cannot create/update API Gateway, IAM roles, or DynamoDB tables.

Q4: What is the AWS Glue Crawler? How does it differ from MSCK REPAIR TABLE for partition discovery?

Ans: Glue Crawler: automatically scans S3, infers schema, and updates Glue Catalog with tables and partitions — supports complex schemas and multiple formats. MSCK REPAIR TABLE: only discovers partitions in Hive-style folder naming (key=value) for an existing Athena table. Crawler is more powerful and automated; MSCK is simpler for Hive-partitioned data.

Q5: What is a SAM template? How does it relate to a CloudFormation template?

Ans: A SAM template is a CloudFormation template with the Transform: AWS::Serverless-2016-10-31 declaration. It adds SAM-specific resource types (AWS::Serverless::Function, AWS::Serverless::Api) that SAM transforms into full CloudFormation resources during deployment. Every SAM template is a valid CloudFormation template after transformation.

Q6: What is AWS Cognito User Pool vs Identity Pool? When would you use each?

Ans: User Pool: manages user sign-up, sign-in, and authentication (username/password, social login, MFA) — returns JWT tokens. Use for app authentication. Identity Pool: exchanges authenticated credentials (from User Pool, Google, etc.) for temporary AWS credentials — lets

users access AWS services directly. Use when users need to call AWS APIs (S3, DynamoDB) from their app.

Q7: What is X-Ray tracing? What is a trace, segment, and subsegment?

Ans: X-Ray tracing records the path of requests through your application. Trace: end-to-end record of one request across all services. Segment: contribution from one service (e.g., Lambda function execution). Subsegment: finer-grained breakdown within a segment (e.g., individual DynamoDB call, external HTTP request).

Q8: What is SKIP.HEADER.LINE.COUNT in an Athena table definition?

Ans: SKIP.HEADER.LINE.COUNT is a SerDe property that tells Athena to skip the first N lines of each CSV file (usually 1 for the header row). Without it, Athena would try to parse the column header line as a data row, causing data type errors.

Practical 7 & 8 — CI/CD Pipeline on AWS (CodeBuild + CodeDeploy + CodePipeline)

7A · Core / Conceptual Questions

Q1: What is a CI/CD pipeline? What do CI and CD stand for?

Ans: CI = Continuous Integration: automatically building and testing code when developers push changes. CD = Continuous Delivery/Deployment: automatically deploying tested code to staging or production. A CI/CD pipeline automates the entire software delivery process from code commit to deployment.

Q2: What is AWS CodeCommit? How does it compare to GitHub?

Ans: CodeCommit is AWS's fully managed, private Git repository service. Like GitHub, it supports Git workflows (clone, push, pull, branches, PRs). Key difference: CodeCommit is private by default, integrates natively with AWS IAM for access control, and keeps code within AWS — better for regulated industries.

Q3: What is AWS CodeBuild? What file does it use to define build steps?

Ans: CodeBuild is a fully managed build service that compiles code, runs tests, and produces deployment artifacts. It uses buildspec.yml (placed in the repository root) to define the build phases, environment variables, and artifact output.

Q4: What is a buildspec.yml file? Name its main sections.

Ans: buildspec.yml defines the CodeBuild build process. Main sections: version (required, e.g., 0.2), env (environment variables), phases (install, pre_build, build, post_build), artifacts (files to output), cache (for faster builds).

Q5: What is AWS CodeDeploy? What is an appspec.yml file?

Ans: CodeDeploy automates application deployments to EC2, Lambda, or ECS. appspec.yml is the deployment configuration file that defines: files to copy (source → destination), permissions, and lifecycle event hooks (scripts to run at each deployment phase).

Q6: What is AWS CodePipeline? How does it orchestrate CodeCommit, CodeBuild, and CodeDeploy?

Ans: CodePipeline is the CI/CD orchestration service. It creates a pipeline with stages: Source stage (pulls code from CodeCommit on push), Build stage (triggers CodeBuild to compile/test/package), Deploy stage (triggers CodeDeploy to deploy to EC2). Each stage's output feeds the next automatically.

Q7: What deployment target was used in this practical for CodeDeploy?

Ans: EC2 instances were used as the deployment target. CodeDeploy deploys the built application (from S3 artifact) to EC2 instances tagged or grouped in a deployment group running the CodeDeploy agent.

Q8: What IAM role is needed on the EC2 instance for CodeDeploy?

Ans: The EC2 instance needs an IAM instance profile with the AmazonEC2RoleforAWSCodeDeploy policy (or equivalent S3 read permissions). This allows the CodeDeploy agent on EC2 to download deployment artifacts from the S3 bucket.

Q9: What is the CodeDeployServiceRole used for?

Ans: The CodeDeployServiceRole is an IAM role assumed by CodeDeploy itself (the service). It grants CodeDeploy permissions to: describe EC2 instances, create deployment lifecycle events, access Auto Scaling groups, and interact with other AWS services during deployment orchestration.

Q10: What S3 bucket is used in this pipeline and what is stored there?

Ans: An S3 bucket serves as the pipeline artifact store. CodeBuild uploads the build artifact (ZIP of built application files) to S3, and CodeDeploy downloads it from S3 to deploy onto EC2 instances. CodePipeline also uses it to pass artifacts between stages.

Q11: What is a deployment group in CodeDeploy?

Ans: A deployment group is a set of EC2 instances (or Lambda functions/ECS services) that CodeDeploy deploys to. Instances are identified by EC2 tags or Auto Scaling group membership. You can have multiple deployment groups per application (e.g., dev, staging, prod).

Q12: What are the lifecycle hooks in an appspec.yml file?

Ans: Lifecycle hooks for EC2: ApplicationStop, DownloadBundle, BeforeInstall, Install, AfterInstall, ApplicationStart, ValidateService. You attach shell scripts to these hooks to customize behavior at each phase (e.g., stop app before install, start app after install, run health check at ValidateService).

Q13: What is a React application? How is it built for production?

Ans: React is a JavaScript library for building user interfaces. For production build: npm run build (or yarn build) runs webpack/Vite to transpile JSX, bundle and minify JavaScript/CSS, and output optimized static files in the build/ or dist/ directory ready for serving from a web server.

Q14: What is a build artifact in CodeBuild? Where is it uploaded?

Ans: A build artifact is the output of the CodeBuild process — typically a ZIP file containing compiled code, static assets, or binaries. CodeBuild uploads artifacts to the S3 artifact bucket defined in the pipeline, from where CodeDeploy retrieves them for deployment.

Q15: What GitHub branch triggers the CI/CD pipeline?

Ans: The main (or master) branch typically triggers the pipeline. In CodePipeline with a GitHub source action, a push to the specified branch (usually main) automatically triggers the Source stage, which then kicks off the Build and Deploy stages.

7B · Advanced Questions

Q1: What is the difference between CodeDeploy in-place deployment and blue/green deployment?

Ans: In-place: application is stopped, new version deployed, then started on the same instances — causes downtime. Blue/Green: new instances (green) are provisioned alongside existing (blue), traffic switches to green after validation — zero downtime. Blue instances are terminated after successful switch.

Q2: What is a CodeBuild service role and what permissions does it typically need?

Ans: The CodeBuild service role is assumed by CodeBuild during builds. Typical permissions: S3 (read source, write artifacts), CloudWatch Logs (write build logs), ECR (pull Docker images), Secrets Manager (read secrets), and any service the build process interacts with (e.g., npm registry, test infrastructure).

Q3: What is GitHub Actions? How does it compare to AWS CodePipeline?

Ans: GitHub Actions is GitHub's native CI/CD platform using workflow YAML files in the repository. Comparison: GitHub Actions integrates tightly with GitHub PRs/branches and has a vast marketplace of actions. CodePipeline integrates deeply with AWS services, IAM, and supports EC2/ECS/Lambda deployments natively. CodePipeline is better for AWS-only stacks; GitHub Actions is more flexible for multi-cloud or open-source.

Q4: What is semantic versioning in build artifacts? Why is it useful?

Ans: Semantic versioning uses MAJOR.MINOR.PATCH format (e.g., 1.2.3). Useful for: tracking exactly which code version is deployed, enabling rollback to a specific version, communicating breaking changes (major), new features (minor), and bug fixes (patch) to stakeholders.

Q5: What does the AmazonEC2RoleforAWSCodeDeploy policy allow the EC2 instance to do?

Ans: It grants EC2 instances permission to: download deployment bundles from S3 (s3:GetObject, s3:GetObjectVersion), list S3 bucket contents (s3:ListBucket), and interact with CodeDeploy service endpoints — all required for the CodeDeploy agent to retrieve and execute deployments.

Q6: What is a CodeDeploy deployment configuration (e.g., OneAtATime, AllAtOnce)?

Ans: Deployment configurations control how CodeDeploy rolls out changes: AllAtOnce — deploys to all instances simultaneously (fastest, highest risk); HalfAtATime — deploys to 50% at a time; OneAtATime — deploys to one instance at a time (slowest, safest, maintains availability). Custom configs can define specific percentages.

Q7: What happens in the build if the buildspec.yml file is missing from the repository?

Ans: CodeBuild fails with a BUILD_FAILED status and logs an error like 'no buildspec.yml found'. The build cannot proceed without knowing the build phases and artifact configuration. The entire pipeline stops at the Build stage.

Q8: What is the purpose of the INSTALL phase in CodeBuild's buildspec lifecycle?

Ans: The INSTALL phase runs first — typically used to install build tools, language runtimes, or system dependencies not available in the build image (e.g., npm install -g specific tools, apt-get install packages). Keep it minimal as it runs on every build.

General AWS Cloud Computing Questions (All Practicals)

General Questions

Q1: What is the AWS Shared Responsibility Model? Give two examples of AWS responsibility and two of customer responsibility.

Ans: AWS manages security OF the cloud; customers manage security IN the cloud. AWS responsibility: physical data center security, hardware maintenance, hypervisor patching, network infrastructure. Customer responsibility: EC2 OS patching, application code security, IAM user management, encryption of data at rest.

Q2: What is the difference between a Region and an Availability Zone in AWS?

Ans: Region: a geographic area containing multiple data centers (e.g., us-east-1 = Northern Virginia). Availability Zone (AZ): one or more discrete data centers within a Region with independent power, cooling, and networking. Regions have multiple AZs (usually 3+) for fault isolation and high availability.

Q3: What is the AWS Free Tier? Which services were used in these practicals under the free tier?

Ans: AWS Free Tier provides limited free usage for 12 months (new accounts) or always-free tiers. Services with free tier used in practicals: Lambda (1M requests/month), DynamoDB (25 GB storage), S3 (5 GB), API Gateway (1M calls), CloudWatch (basic monitoring), EC2 (750 hrs t2.micro), RDS (750 hrs db.t2.micro).

Q4: What is the AWS Management Console? What are the alternatives for interacting with AWS?

Ans: The AWS Management Console is the web-based GUI for managing AWS services. Alternatives: AWS CLI (command-line interface), SDKs (boto3 for Python, AWS SDK for Java/Node.js/etc.), CloudFormation/Terraform (IaC), AWS CDK (code-based IaC), REST APIs (direct HTTP calls).

Q5: What is an AWS ARN (Amazon Resource Name)? Give an example.

Ans: An ARN uniquely identifies any AWS resource globally. Format: arn:partition:service:region:account-id:resource. Example: arn:aws:iam::123456789012:user/JohnDoe or arn:aws:s3:::my-bucket or arn:aws:lambda:us-east-1:123456789012:function:MyFunction.

Q6: What is the difference between authentication and authorization in AWS IAM?

Ans: Authentication: verifying identity — who are you? (done via access keys, passwords, MFA). Authorization: determining permissions — what can you do? (done via IAM policies defining allowed/denied actions on resources). Both are required for secure access.

Q7: What is encryption at rest vs encryption in transit? Give one AWS service example of each.

Ans: Encryption at rest: data encrypted while stored on disk using keys (KMS). Example: RDS storage encryption with KMS, S3 SSE-S3. Encryption in transit: data encrypted while moving over networks using TLS/SSL. Example: HTTPS for API Gateway, SSL connections to RDS.

Q8: What is auto-scaling in AWS? Which practical's architecture would benefit most from it?

Ans: Auto-scaling automatically adjusts the number of EC2 instances (or Lambda concurrency, DynamoDB capacity) based on demand. Practical 7 (CI/CD with EC2 + CodeDeploy) benefits most — adding Auto Scaling Groups would allow the web application to scale EC2 instances based on traffic load.

Q9: What is the difference between horizontal and vertical scaling?

Ans: Horizontal scaling (scale out/in): adding or removing more instances/servers to distribute load — preferred in cloud for resilience. Vertical scaling (scale up/down): increasing or decreasing the size (CPU, RAM) of a single instance — limited by maximum instance size and requires downtime.

Q10: What is serverless computing? Name three AWS serverless services used in these practicals.

Ans: Serverless: you write code without managing servers; AWS handles infrastructure, scaling, and availability; you pay only for what you use. Serverless services from practicals: Lambda (compute), API Gateway (HTTP API), DynamoDB (database), Athena (queries), Step Functions (orchestration), S3 (storage).

Q11: What is the purpose of tagging AWS resources?

Ans: Tags (key-value pairs on resources) serve multiple purposes: cost allocation and billing reports (track costs by project/team), automation (trigger scripts based on tags), access control (IAM policies based on tags), organization (filter/group resources in console), and compliance/governance.

Q12: What is cost optimization in AWS? Name two strategies used across these practicals.

Ans: Cost optimization means achieving the lowest price for desired performance. Strategies used: (1) Serverless services (Lambda, DynamoDB On-Demand, Athena) — pay only per request, no idle costs; (2) S3 for storage — using S3 as cheap durable storage for artifacts, logs, and Athena data instead of expensive compute storage; (3) Free Tier usage for learning environments.